COMPUTER LABORATORY – I

Assignment 2

Data Loading, Storage and File Formats

A simple, detailed code explanation + theory + viva cross-question bank

Experiment focus	Load sales records from CSV, Excel and JSON; combine, clean, analyze, visualize and export.
Python libraries	Pandas, Matplotlib, pathlib
Notebook result	9,994 cleaned rows and 21 original columns, with two derived columns for analysis.
Prepared from	Uploaded Assignment_2.ipynb and CL1 assignment schedule.

How to use this guide: First understand the overall workflow, then learn what each code block does, and finally practice the viva questions aloud. Answers are intentionally written in straightforward language.

Note: The explanations below distinguish what your notebook actually does from optional improvements. Do not claim that an improvement is already present in your submitted code.

1. Understand the experiment

The problem statement asks us to analyze sales data that is stored in multiple file formats. In real projects, data may arrive from different systems: one team may send a CSV file, another may maintain an Excel workbook, and an application may export JSON. The goal is to bring these records into one consistent structure and analyze them.

Workflow of your notebook

1. Load	Read CSV, Excel and JSON into separate Pandas DataFrames.
2. Inspect	Check dimensions, preview rows and count missing values.
3. Combine	Stack the three datasets row-wise into one DataFrame.
4. Clean	Standardize text, convert types, remove duplicate and incomplete rows.
5. Transform	Create Unit Price and Order Month columns.
6. Analyze	Calculate descriptive statistics, total sales, average order value and category sales.
7. Visualize	Draw category-wise and month-wise sales charts.
8. Export	Save the cleaned result as a CSV file in outputs/.

What is a DataFrame?

A DataFrame is a two-dimensional table in Pandas. It has rows and columns, similar to an Excel sheet or a database table. Each column can contain a different type of value, such as dates, text or numbers. In this experiment, **sales** is the final combined DataFrame.

What are the three file formats?

• **CSV (Comma-Separated Values):** a plain-text table where values are separated by commas. It is lightweight and widely supported.

• **Excel:** spreadsheet format such as .xlsx. It can contain sheets, formatting and formulas.

• **JSON (JavaScript Object Notation):** a text format commonly used for exchanging structured data between applications and APIs.

Important distinction: CSV, Excel and JSON are storage/exchange formats. A Pandas DataFrame is the in-memory structure used by Python for analysis.

2. Code walkthrough: loading and inspecting

Imports and reading the three files

```
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

csv_data = pd.read_csv('sales_part1.csv')
excel_data = pd.read_excel('sales_part2.xlsx')
json_data = pd.read_json('sales_part3.json')
```

import pandas as pd loads Pandas and gives it the short alias **pd**. Pandas provides tools to read, clean, transform, group and analyze tabular data.

import matplotlib.pyplot as plt loads the plotting interface of Matplotlib. The alias **plt** is used later for chart formatting and display.

from pathlib import Path imports Path, a modern way to work with file and folder paths. It is used when creating the output folder.

pd.read_csv() reads a CSV file and returns a DataFrame. **pd.read_excel()** reads an Excel worksheet. **pd.read_json()** reads JSON data. Each result is stored in a separate variable.

Inspecting each dataset

```
for data in [csv_data, excel_data, json_data]:
    print('Shape:', data.shape)
    display(data.head(2))
    display(data.isna().sum())
```

The list contains the three DataFrames. The **for** loop runs the same inspection steps on each one, avoiding repeated code.

• **data.shape** returns a tuple: (number of rows, number of columns). In the notebook, each file initially has 3,600 rows and 21 columns.

• **data.head(2)** displays the first two rows. It helps check whether columns and values were read correctly.

• **data.isna()** creates a True/False table showing missing cells. **.sum()** counts True values column-wise.

• **display()** is commonly available in Jupyter Notebook and renders tables neatly. It is not a Pandas method.

Inspection result: the CSV and Excel inputs show one missing Postal Code each. The JSON input has 806 missing values in many columns, indicating that some rows are largely empty. This is why inspection should happen before cleaning.

Viva questions

Q. Why inspect data before processing it?
Answer: To understand its size, columns, sample values and missing data. It helps us choose suitable cleaning steps.

Q. What does shape return?
Answer: A tuple (rows, columns). For example, (3600, 21) means 3,600 records and 21 columns.

Q. What is the difference between head() and tail()?
Answer: head() shows the first rows; tail() shows the last rows. Both show five rows by default.

Q. What does isna().sum() do?
Answer: It counts missing values in each column.

3. Code walkthrough: merging and cleaning

Combine the three sources

```
sales = pd.concat(
    [csv_data, excel_data, json_data],
    ignore_index=True
)
```

pd.concat() concatenates objects. Here, the DataFrames are stacked vertically (row-wise) because they have the same columns. **ignore_index=True** creates a fresh index from 0 onward instead of preserving the original index values from each file.

The three files contain 3,600 rows each, so the combined table initially has 10,800 rows. Some rows represent duplicate or incomplete records.

Standardize column names and text

```
sales.columns = sales.columns.str.strip()

for column in ['Region', 'Category', 'Sub-Category']:
    sales[column] = sales[column].str.strip().str.title()
```

sales.columns accesses column labels. **.str** applies string operations to every label. **strip()** removes leading and trailing spaces. This prevents accidental differences such as *'Region'* and *' Region '*.

The loop applies the same text cleanup to Region, Category and Sub-Category. **str.title()** converts text to title case, e.g. 'office supplies' becomes 'Office Supplies'.

Convert columns to suitable data types

```
sales['Order Date'] = pd.to_datetime(
    sales['Order Date'], errors='coerce'
)
sales['Sales'] = pd.to_numeric(
    sales['Sales'], errors='coerce'
)
sales['Quantity'] = pd.to_numeric(
    sales['Quantity'], errors='coerce'
)
```

pd.to_datetime() converts date-like values to Pandas datetime values. **pd.to_numeric()** converts values to numeric form. With **errors='coerce'**, invalid values become missing values (NaT for dates, NaN for numbers) rather than stopping execution.

Remove duplicates and incomplete essential records

```
sales = sales.drop_duplicates(subset='Row ID')
sales = sales.dropna(
    subset=['Order ID', 'Order Date', 'Sales', 'Quantity']
)
sales['Category'] = sales['Category'].fillna('Unknown')
sales['Region'] = sales['Region'].fillna('Unknown')
print('Cleaned shape:', sales.shape)
```

drop_duplicates(subset='Row ID') keeps one record for each Row ID. **dropna(subset=...)** removes rows missing any of the listed essential fields. **fillna('Unknown')** replaces missing category or region labels with a meaningful placeholder.

The output is **(9994, 21)**. Starting with 10,800 combined rows, 806 duplicate Row IDs are removed, leaving 9,994. In this particular data, the subsequent required-field drop does not reduce the row count further.

4. Data cleaning theory and important distinctions

Missing values: NaN and NaT

A missing value means that a field has no usable value. Pandas commonly represents missing numeric/object values as NaN and missing datetime values as NaT. Missing values can occur because a source system did not collect a field, data was lost, or an incomplete record was exported.

• **Drop rows:** appropriate when a required field is missing and the record cannot be used.

• **Fill with a constant:** useful for categorical labels, such as 'Unknown'.

• **Fill with mean/median/mode:** sometimes used for numeric or categorical columns, but the choice should match the meaning and distribution of the data.

• **Keep missing:** sometimes correct when missingness itself is meaningful or the field is not needed.

Duplicate records

A duplicate is a repeated record. In this notebook, Row ID is treated as the identifier for a row. Removing duplicates using a key is safer than comparing every column, provided the key is genuinely unique.

Data type conversion

A date stored as text cannot be reliably used for date arithmetic until converted to datetime. Numeric-looking text should be converted before calculations. Using errors='coerce' makes invalid values visible as missing values, which should then be checked.

Why standardization matters

Values such as 'west', 'West ' and 'WEST' may be treated as different categories. Trimming spaces and standardizing case makes grouping and counting more reliable. Standardization should not alter meaningful distinctions in the original data.

Theory viva questions

Q. What is data cleaning?
Answer: The process of identifying and correcting or handling missing, duplicate, inconsistent, invalid or incorrectly formatted data.

Q. What is the difference between dropna() and fillna()?
Answer: dropna() removes rows or columns containing missing values according to its settings. fillna() replaces missing values with a chosen value or method.

Q. What is an outlier?
Answer: A value unusually far from most observations. It may be a genuine rare event or a data-entry error; it should be investigated, not automatically deleted.

Q. What is type casting?
Answer: Converting a value or column from one data type to another, for example text to numeric or text to datetime.

Q. Why use errors='coerce'?
Answer: Invalid values are converted to missing values, allowing processing to continue. We must inspect and handle those new missing values afterwards.

Q. Does fillna('Unknown') calculate the real missing category?
Answer: No. It only assigns a placeholder. It does not infer the actual category.

5. Feature creation and analysis

Create two derived columns

```
sales['Unit Price'] = sales['Sales'].div(
    sales['Quantity'].replace(0, float('nan'))
)
sales['Order Month'] = sales['Order Date'].dt.month.astype(str)
```

Sales.div(...) performs element-wise division. Quantity values of zero are first replaced with NaN to avoid division by zero. Therefore, Unit Price is calculated as Sales divided by Quantity for each row.

Order Date.dt.month extracts the month number (1 to 12) from a datetime column. **astype(str)** converts that number to text.

Important: this derived Unit Price is the row's sales amount per unit. It should not automatically be described as the original list price because discounts and other pricing effects may already be reflected in Sales.

Descriptive statistics and totals

```
display(sales[['Sales', 'Quantity', 'Unit Price']].describe())
print('Total sales:', round(sales['Sales'].sum(), 2))

order_totals = sales.groupby('Order ID')['Sales'].sum()
print('Average order value:', round(order_totals.mean(), 2))

category_sales = sales.groupby('Category')['Sales'].sum()
display(category_sales)
display(sales['Category'].value_counts())
```

• **describe()** gives count, mean, standard deviation, minimum, quartiles and maximum for numeric columns.

• **sum()** adds values. **round(value, 2)** rounds a result to two decimal places.

• **groupby('Order ID')** creates groups for each order. **['Sales'].sum()** calculates the total sales in each order.

• **mean()** calculates the arithmetic average of the order totals. This is the average order value (AOV) used in this notebook.

• **groupby('Category')['Sales'].sum()** calculates total sales for each product category.

• **value_counts()** counts how many rows belong to each category. It measures record frequency, not sales amount.

Results produced by the notebook

Cleaned rows	9,994
Total sales	2,297,200.86
Average order value	458.61
Furniture sales	741,999.80
Office Supplies sales	719,047.03
Technology sales	836,154.03

Technology has the highest total sales among the three categories in this output. Office Supplies has the highest number of rows (6,026), which shows why frequency and sales value are different metrics.

6. Visualization and exporting the result

Category sales bar chart

```
category_sales.plot(
    kind='bar',
    ylabel='Sales',
    title='Sales by category',
    rot=0
)
plt.tight_layout()
plt.show()
```

plot(kind='bar') draws a bar chart from the category sales Series. Each bar represents one category; its height represents total sales. **ylabel** labels the vertical axis, **title** adds a chart title and **rot=0** keeps category labels horizontal.

plt.tight_layout() adjusts spacing so labels and titles are less likely to overlap. **plt.show()** displays the chart.

Monthly sales chart

```
sales.groupby('Order Month')['Sales'].sum().plot(
    figsize=(10, 4),
    xlabel='Month',
    ylabel='Sales',
    title='Monthly sales'
)
plt.tight_layout()
plt.show()
```

This groups rows by the derived Order Month and sums Sales for each month. **figsize=(10, 4)** sets the figure size in inches. The plot uses the default line style because no kind is specified.

Potential issue: Order Month was converted to a string. String sorting may put months in the order 1, 10, 11, 12, 2, 3, and so on. A better approach is to group using the numeric month or use a year-month period, especially when the dataset covers multiple years.

Export cleaned data

```
Path('outputs').mkdir(exist_ok=True)
sales.to_csv('outputs/02_cleaned_sales.csv', index=False)
```

Path('outputs') represents the output folder. **mkdir(exist_ok=True)** creates it if needed and does not raise an error if it already exists. **to_csv()** writes the DataFrame to a CSV file. **index=False** prevents Pandas from writing the DataFrame index as an extra column.

Visualization and export viva

Q. Why use a bar chart?
Answer: It makes it easy to compare values across discrete categories.

Q. What is the difference between a bar chart and a line chart?
Answer: A bar chart compares categories. A line chart is useful for showing ordered or continuous trends, often over time.

Q. Why use tight_layout()?
Answer: It adjusts spacing to reduce overlap between chart elements.

Q. What does index=False do in to_csv()?
Answer: It prevents the DataFrame row index from being saved as a separate column.

Q. What is the difference between read_csv() and to_csv()?
Answer: read_csv() loads CSV data into a DataFrame; to_csv() writes a DataFrame to a CSV file.

7. Functions used: quick reference + code cross-questions

pd.read_csv(path)	Reads CSV data into a DataFrame.
pd.read_excel(path)	Reads an Excel file into a DataFrame.
pd.read_json(path)	Reads JSON data into a DataFrame.
pd.concat(list, ignore_index=True)	Combines DataFrames; here, stacks rows.
shape	Returns (rows, columns). It is an attribute, not a function.
head(n)	Returns the first n rows.
isna()	Marks missing values as True.
sum()	Adds numeric values or counts True values column-wise.
str.strip()	Removes whitespace at the start and end of strings.
str.title()	Converts strings to title case.
pd.to_datetime()	Converts values to datetime.
pd.to_numeric()	Converts values to numeric.
drop_duplicates()	Removes duplicate rows based on selected columns.
dropna() / fillna()	Remove missing records / replace missing values.
div()	Element-wise division for Pandas objects.
astype(str)	Converts values to strings.
describe()	Returns descriptive statistics.
groupby()	Groups rows by one or more keys.
value_counts()	Counts occurrences of unique values.
plot()	Creates a plot from Series or DataFrame data.
mkdir()	Creates a directory.
to_csv()	Exports a DataFrame to CSV.

Likely code-level cross-questions

Q. Why use a loop for the three DataFrames?
Answer: The same inspection and cleaning operation can be repeated without writing it three times.

Q. Why is ignore_index=True used in concat?
Answer: It creates a continuous new index after combining the inputs.

Q. Is shape a method?
Answer: No. It is an attribute, so we write data.shape, not data.shape().

Q. What does .dt do?
Answer: It provides datetime-specific properties and operations, such as extracting year, month or day.

Q. Why replace zero quantity before division?
Answer: Division by zero is undefined. Replacing zero with NaN avoids an infinite or invalid unit-price result.

Q. What is a Series?
Answer: A one-dimensional labeled Pandas structure, such as the category_sales result.

8. Full viva practice: theory, reasoning and improvements

Q. What is the objective of this experiment?
Answer: To load sales data from CSV, Excel and JSON, combine it, clean it, analyze sales patterns, visualize results and save a cleaned output.

Q. Why do we use Pandas instead of manually reading files?
Answer: Pandas provides convenient functions for loading, filtering, cleaning, grouping and analyzing tabular data.

Q. What is ETL?
Answer: Extract, Transform, Load. Extract means obtaining data from sources; transform means cleaning or changing it; load means storing it in a destination. This experiment follows a simple ETL-like workflow.

Q. What is data wrangling?
Answer: Preparing raw data for analysis by cleaning, reshaping, combining and transforming it.

Q. What is the difference between mean and median?
Answer: Mean is the sum divided by the count. Median is the middle value after sorting and is less affected by extreme values.

Q. What does standard deviation tell us?
Answer: It measures how spread out values are around the mean. A larger standard deviation indicates greater spread.

Q. What is a quartile?
Answer: A value that divides sorted data into four equal parts. describe() reports the 25th, 50th and 75th percentiles.

Q. What is aggregation?
Answer: Combining multiple records into a summary, such as total sales per category or per region.

Q. What is average order value in this notebook?
Answer: First sum sales for each Order ID, then calculate the mean of those order totals.

Q. Why can value_counts() and groupby().sum() give different rankings?
Answer: value_counts() counts records; groupby().sum() totals the Sales values. A category can have many low-value rows but lower total sales.

Q. Why is JSON popular for APIs?
Answer: It is lightweight, readable, language-independent and represents structured data using objects and arrays.

Q. What is the difference between CSV and JSON?
Answer: CSV represents tabular data as rows and columns; JSON represents nested objects and arrays and is common in web APIs.

Improvements you can mention if the examiner asks

• **Check missing values after conversion:** errors='coerce' can create new NaN/NaT values. Count them again before deciding how to handle them.

• **Validate duplicates:** confirm that Row ID is a unique identifier before using it as the duplicate key.

• **Preserve month order:** keep month numeric, or group by a year-month Period so months are chronological and different years are not mixed.

• **Handle zero quantity explicitly:** inspect rows where Quantity is zero and decide whether to retain or exclude their Unit Price.

• **Use a date-aware trend:** aggregate by month and year rather than month number alone for multi-year sales data.

• **Check the output:** reload the exported CSV and verify its row count, columns and data types.

One-sentence conclusion for viva: “I integrated sales records from three file formats using Pandas, cleaned and standardized the data, derived useful fields, summarized sales by order and category, visualized category and monthly trends, and exported the cleaned dataset for future analysis.”

Source boundary: This guide explains the code and results in your uploaded notebook. The additional theory and improvement suggestions are general explanations to help you answer follow-up questions; they are not extra steps already executed in the notebook.